

Cap HTB – From IDOR to Root via Linux Capabilities

Introduction

The retired HackTheBox machine **Cap** is a great example of how seemingly small misconfigurations can combine into a complete compromise. This box highlights two important security concepts:

1. **Insecure Direct Object Reference (IDOR)** in a web application.
2. **Abused Linux capabilities** for privilege escalation.

Together, they illustrate why proper access control and least privilege hardening are essential.

Vulnerability #1: Insecure Direct Object Reference (IDOR)

An **IDOR** occurs when a web application exposes direct references to objects (like files or records) without properly checking if the user has permission to access them.

In Cap, the web dashboard let users generate packet captures via the URL:

`http://<ip>/data/<id>`

The problem: the server did not verify ownership of the capture. By incrementing or decrementing the id, we could retrieve other users' captures. This is a textbook IDOR issue.

Real-World Context:

IDOR has been one of the OWASP Top 10 vulnerabilities for years. It has real-world CVEs such as:

- **CVE-2021-41349 (Zoho ManageEngine)** – IDOR allowed unauthorized data access.
- **CVE-2019-14956 (WordPress plugin)** – IDOR exposed sensitive user files.

On Cap, this weakness let us download a packet capture that contained plaintext FTP credentials.

Vulnerability #2: Plaintext Credentials

Inside the PCAP file, we found FTP authentication traffic:

USER nathan

PASS Buck3tH4TF0RM3!

Because FTP is unencrypted, these credentials were fully exposed. While this was a deliberate CTF setup, it mirrors real-world risks when organizations still rely on insecure protocols like FTP or Telnet.

Those credentials worked not only for FTP, but also for **SSH**, giving us a foothold as the user nathan.

Vulnerability #3: Misconfigured Linux Capabilities

Once inside the machine, the next step was privilege escalation. We checked for binaries with special capabilities:

```
getcap -r / 2>/dev/null
```

Output showed:

```
/usr/bin/python3.8 = cap_setuid,cap_net_bind_service+eip
```

The cap_setuid capability is powerful: it allows a process to change its user ID without needing the SUID bit. This effectively gave Python permission to spawn processes as **root**.

By running:

```
python3.8 -c 'import os; os.setuid(0); os.system("/bin/bash")'
```

we instantly escalated to root.

Lessons Learned

1. **Never trust user-controlled IDs** – Always enforce access controls server-side when referencing files or records.
 2. **Deprecate insecure protocols** – FTP sends credentials in cleartext. Use SFTP or FTPS to protect authentication.
 3. **Audit Linux capabilities** – Tools like getcap or [LinPEAS](#) help identify binaries with dangerous capabilities. Least privilege means avoiding giving binaries unnecessary powers.
-

Conclusion

Cap demonstrates how a chain of small issues can lead to full compromise:

- **IDOR** → Unauthorized packet capture.
- **FTP in cleartext** → Credential disclosure.
- **Password reuse (SSH)** → User shell.
- **Python with cap_setuid** → Root.

While simple in isolation, together they form a complete attack path. This is why penetration testers and defenders alike must think in terms of **chaining vulnerabilities**.

Cap may be a retired HTB machine, but the lessons apply directly to real-world systems still vulnerable to IDOR and misconfigured capabilities.